Breaking the Composition Cliff: Mutation-Guided Specification Synthesis for Multi-Function Programs*

Ishraq Tanvir
FOF Business Point
ishraq.tanvir@fofbd.com

Abstract

Large language models (LLMs) can generate code, but synthesizing formal specifications for this code remains challenging. While previous approaches report high verification rates on individual functions, real-world programs are composed of modular interprocedural call graphs. We evaluate six frontier language models on synthesizing formal Dafny specifications for 300 multi-function programs with call-graphs of sizes 2–5 (DAFNYCOMP-300), revealing two key shortcomings. First, we find that models struggle with compositional specifications: while models reach 91.7%–100% specification completeness on individual functions with repair, zero-shot specification success collapses from up to 95.0% on single functions down to 14.0%–41.3% on multi-function call graphs. Second, the traditional repair loop, which relies on compiler error messages, leads models to learn vacuous specifications, e.g., trivial inequalities such as `ensures res >= 0`. While these specifications appear logically sound, they are overly weak and cannot reject incorrect implementations. To address this, we present MUTATION-CEGIS, an approach that uses AST code mutants to test specifications for vacuity. When a candidate specification mistakenly admits a buggy mutant, we extract the code change as a concrete counterexample for the model to refine its specification. On single-function benchmarks (CLOVERBENCH-60), MUTATION-CEGIS boosts specification completeness to 91.7%–100.0% while reducing repair turns by up to 36.8% and saving up to 24.9% of total tokens. On compositional call graphs (DAFNYCOMP-300), MUTATION-CEGIS systematically detects vacuous bounding contracts across model families and, when synthesis succeeds, drives models toward tighter first-order relational specifications ($\forall, \exists$). Finally, we find that for deeper call-graphs, exact specification synthesis results in SMT-solver timeouts, demonstrating that future verification agents will benefit from synthesizing auxiliary lemmas in addition to specifications.

1 Introduction

Deductive program verification under Hoare logic [Hoare, 1969] provides mathematical guarantees of software correctness by proving an implementation's conformance to its formal specification. In verification-aware languages and frameworks such as Dafny [Leino, 2010] and Verus for Rust [Yang et al., 2025], developers write method contracts—preconditions (`requires`), postconditions (`ensures`), and inductive invariants (`invariant`)—which the verifier converts into first-order verification conditions checked by SMT solvers such as Z3 [De Moura and Bjørner, 2008]. While this approach eliminates runtime bugs and functional defects from programs, writing formal specifications manually requires substantial human effort [Sun et al., 2024, First et al., 2023].

*Code and benchmarks available at `https://github.com/devishraq/composition-cliff`.

Workshop on Who Verifies the Agents? Toward Reliable Agent Development, NeurIPS 2026.

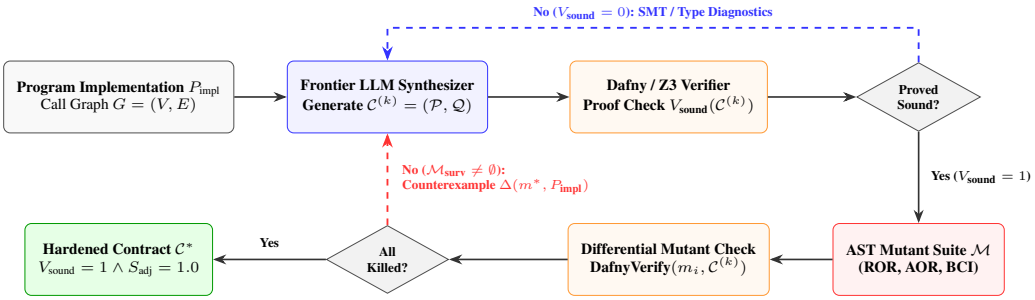


Figure 1: **The MUTATION-CEGIS Architecture.** While compiler-only feedback (MODE 1, blue dashed) terminates on vacuous contracts that verify on P_{impl}, MUTATION-CEGIS (MODE 2, red dashed) tests candidate contracts against an AST mutant matrix $\mathcal{M}$. Surviving mutants generate concrete differential counterexamples injected back into the LLM context to enforce semantic contract tightness.

Recent work has proposed Large Language Models (LLMs) to automatically generate formal specifications [Sun et al., 2024, Wu et al., 2022, First et al., 2023]. However, existing benchmarks for specification synthesis mainly focus on single-function routines [Sun et al., 2024, Austin et al., 2021] whereas real-world software systems are comprised of modular call graphs in which functions pass data to each other while relying on contracts across component boundaries [Xu et al., 2025].

In this paper we evaluate contract synthesis across 6 frontier LLM families on multi-procedure call graphs containing 2-5 interdependent functions (DAFNYCOMP-300) [Xu et al., 2025]. Our experiments reveal two findings:

1. **Composition Cliff:** Model performance drops off dramatically as one transitions from single function call graphs to multiple interprocedure graphs. Zero-shot success falls from 95.0% to 41.3% on Gemini 3.7 Flash, 83.3% to 32.7% on GLM-5.3 Flash, 53.3% to 19.0% on GPT-5.6 Luna, 40.0% to 14.0% on DeepSeek V4 Flash, and 33.3% to 14.7% on Ling-3.0 Flash. While models generate locally consistent annotations for individual procedures in the graph, the annotations fail to respect invariants in caller-callee relationships.
2. **Vacuous Verification Trap:** Automated verifiers check only whether the implementation satisfies the candidate contract ($\mathcal{P} \wedge [\![P]\!] \implies \mathcal{Q}$). In standard compiler-in-the-loop repair loops (MODE 1) [First et al., 2023, Le et al., 2022], models learn to evade verification errors by defaulting to weak, vacuous inequalities (e.g., `ensures res >= 0`). Since the ground-truth code also satisfies the weak bounds, these invariants verify effortlessly without ever detecting a meaningful code mutation.

We solve this problem via MUTATION-CEGIS (Figure 1), framing contract hardening as a Counterexample-Guided Inductive Synthesis problem [Solar-Lezama et al., 2006, Alur et al., 2013]. We evaluate candidate specifications against a set of AST code mutants [DeMillo et al., 1978, Jia and Harman, 2011, Just et al., 2014]. If a mutant passes verification, it provides evidence that the contract is too weak. We transform the surviving mutants into concrete code diffs and feed them back to the model prompt, guiding the LLM to write exact specifications.

Listing 1: Motivating Case Study (2-Node Call Graph). MODE 1 produces a vacuous bound (`ensures res >= 0`) that verifies instantly but fails to kill an AST mutant in `HelperClamp`. MODE 2 injects the mutant diff as a counterexample, forcing the model to synthesize exact conditional equalities.

```
// Node 1: Helper subroutine
method HelperClamp(x: int, max_val: int) returns (c: int)
  requires max_val > 0
  // Mode 1 Vacuous Spec: ensures c >= 0
  // Mode 2 CEGIS Spec:   ensures (x < 0 ==> c == 0) &&
  //                              (0 <= x <= max_val ==> c == x) &&
  //                              (x > max_val ==> c == max_val)
{
  if x < 0 { c := 0; }
```

```
  // Injected AST Mutant (ROR): if x <= 0 { c := 1; }
  else if x > max_val { c := max_val; }
  else { c := x; }
}

// Node 2: Parent procedure invoking HelperClamp
method ClampAndScale(v: int, limit: int) returns (res: int)
  requires limit > 0
  // Mode 1 Vacuous Spec: ensures res >= 0
  // Mode 2 CEGIS Spec:   ensures res == 2 * (if v < 0 then 0
  //                              else if v > limit then limit else v)
{
  var clamped := HelperClamp(v, limit);
  res := clamped * 2;
}
```

Contributions. In summary, this work provides three main contributions: (1) **Characterizing the Composition Cliff:** we report the performance drop (referred to as the composition cliff) of 6 frontier model families across 6,480 task-mode verification runs over DAFNYCOMP-300 [Xu et al., 2025] and CLOVERBENCH-60 [Sun et al., 2024] when transitioning from single procedure verification to multi-procedure call graphs; (2) **Mutation-CEGIS Hardening:** we present an inductive synthesis loop that attains 91.7%–100.0% true specification completeness on single methods (reducing repair turns by up to 36.8% and token usage by 24.9%), and on multi-procedure call graphs systematically eliminates spurious 50-character inequalities and, for Qwen 3.7 Flash, improves hardened multi-procedure synthesis over compiler-only repair ($24.7\% \rightarrow 31.7\%$); and (3) **Specification Over-Strengthening Dilemma:** we observe that synthesis of exact specifications on 4 to 5 node call graphs leads to SMT timeouts (> 15s), suggesting that contract hardening on deep call graphs fundamentally requires automated auxiliary lemma synthesis [Yang et al., 2025].

2 Formal Framework & Mutation-CEGIS

2.1 Verification Semantics & Soundness

Let P_{impl} denote a deterministic program implementation defined over state space Σ. A candidate formal contract $\mathcal{C} = (\mathcal{P}, \mathcal{Q})$ consists of precondition predicate $\mathcal{P} : \Sigma \rightarrow \{\text{true}, \text{false}\}$ and postcondition relation $\mathcal{Q} : \Sigma \times \Sigma \rightarrow \{\text{true}, \text{false}\}$. The contract $\mathcal{C}$ is semantically valid with respect to P_{impl} if:

$$\text{Valid}(P_{\text{impl}}, \mathcal{C}) \iff \forall \sigma, \sigma' \in \Sigma.\, \mathcal{P}(\sigma) \wedge [\![P_{\text{impl}}]\!](\sigma, \sigma') \implies \mathcal{Q}(\sigma, \sigma') \tag{1}$$

We define the binary soundness indicator $V_{\text{sound}}(\mathcal{C}) \in \{0, 1\}$ to denote successful verification by the automated verifier (DafnyVerify$(P_{\text{impl}}, \mathcal{C}) = 1$); successful verification establishes contract validity, while SMT solver timeouts (> 15s) or proof obligation violations are treated as verification failure (0). While $V_{\text{sound}}(\mathcal{C}) = 1$ guarantees partial correctness, it allows trivial contracts such as $\mathcal{Q} \equiv$ true or loose bounds like $\mathcal{Q} \equiv \text{res} \geq 0$.

2.2 Quantifying Specification Completeness via AST Mutation

To measure contract completeness without incurring the syntactic invalidity and redundant mutant explosion of broad operator suites [DeMillo et al., 1978, Offutt et al., 1996, Just et al., 2014], we follow selective mutation testing principles [Offutt et al., 1996] and construct an AST mutant matrix $\mathcal{M} = \{m_1, m_2, \ldots, m_N\}$ using three core semantic operator classes: (1) **Relational Operator Replacement (ROR)**, inverting comparison operators ($\{<, \leq, >, \geq, ==, !=\}$); (2) **Arithmetic Operator Replacement (AOR)**, mutating arithmetic expressions ($\{+, -, *, /, \%\}$); and (3) **Boundary Condition Inversion (BCI)**, perturbing constants by off-by-one shifts ($c \mapsto c \pm 1$).

A contract $\mathcal{C}$ kills mutant $m_i \in \mathcal{M}$ if Dafny proves that m_i violates $\mathcal{C}$, written Killed$(m_i, \mathcal{C}) \iff \neg$DafnyVerify$(m_i, \mathcal{C})$. To exclude semantically equivalent mutants that cannot be rejected by any sound specification, we define the subset of distinguishable mutants:

$$\mathcal{M}_{\text{kill}} = \{m \in \mathcal{M} \mid \text{DafnyVerify}(m, \mathcal{C}_{\text{GT}}) = \text{False}\} \tag{2}$$

The *Raw Mutation Score* S_{raw} and *Adjusted Mutation Score* S_{adj} relative to ground-truth contract $\mathcal{C}_{\text{GT}}$ are defined as:

$$S_{\text{raw}}(\mathcal{C}, \mathcal{M}) = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \mathbb{I}(\text{Killed}(m, \mathcal{C})), \qquad S_{\text{adj}}(\mathcal{C}, \mathcal{M}) = \frac{\sum_{m \in \mathcal{M}} \mathbb{I}(\text{Killed}(m, \mathcal{C}))}{\sum_{m \in \mathcal{M}} \mathbb{I}(\text{Killed}(m, \mathcal{C}_{\text{GT}}))} \tag{3}$$

Definition 1 (Hardened Contract)**.** *A synthesized contract $\mathcal{C}$ is hardened if it verifies on P_{impl} and kills every non-equivalent mutant detected by the ground-truth contract:*

$$\textit{Hardened}(\mathcal{C}) \iff V_{sound}(\mathcal{C}) = 1 \ \wedge \ S_{adj}(\mathcal{C}, \mathcal{M}) = 1.000 \tag{4}$$

Thus, $S_{\text{adj}} = 1.0$ establishes specification completeness with respect to the selected killable mutant suite $\mathcal{M}_{\text{kill}}$ (guaranteeing that no non-equivalent mutant detected by $\mathcal{C}_{\text{GT}}$ survives), rather than absolute semantic equivalence.

2.3 The Mutation-Guided CEGIS Algorithm

Algorithm 1 formalizes the MUTATION-CEGIS hardening loop. The synthesizer initializes prompt context $T^{(1)}$ and iterates over discrete synthesis turns $k \in \{1, \ldots, K_{\max}\}$. In each turn, the model generates candidate contract $\mathcal{C}^{(k)}$. If Dafny rejects the contract on P_{impl} ($v = \text{False}$), diagnostic error messages E_{diag} are injected to repair syntax or proof obligations (MODE 1). Once soundness is established ($v = \text{True}$), the verifier executes candidate $\mathcal{C}^{(k)}$ against distinguishable mutant suite $\mathcal{M}_{\text{kill}}$. If no mutants survive ($\mathcal{M}_{\text{surv}} = \emptyset$), the contract is guaranteed sound and hardened ($S_{\text{adj}} = 1.0$). Otherwise, a surviving mutant $m^* \in \mathcal{M}_{\text{surv}}$ is converted into an AST differential counterexample $\Delta(m^*, P_{\text{impl}})$ and fed back into the context to enforce exact postcondition tightness (MODE 2).

Algorithm 1 MUTATION-CEGIS Contract Hardening

Require: Program implementation P_{impl}, distinguishable mutant suite $\mathcal{M}_{\text{kill}}$, repair budget $K_{\max}$, LLM synthesizer $\mathcal{M}_{\text{LLM}}$
Ensure: Hardened contract $\mathcal{C}^*$ or verification failure $\perp$

$T^{(1)} \leftarrow \text{Prompt}_{\text{initial}}(P_{\text{impl}})$
for $k = 1$ **to** $K_{\max}$ **do**
  $\mathcal{C}^{(k)} \leftarrow \mathcal{M}_{\text{LLM}}(T^{(k)})$ {Synthesize candidate contract $(\mathcal{P}, \mathcal{Q})$}
  $(v, E_{\text{diag}}) \leftarrow \text{DafnyVerify}(P_{\text{impl}}, \mathcal{C}^{(k)})$ {Mode 1: Soundness on P_{impl}}
  if $v = \text{False}$ **then**
    $T^{(k+1)} \leftarrow \text{Prompt}_{\text{repair}}(P_{\text{impl}}, \mathcal{C}^{(k)}, E_{\text{diag}})$ {Compiler Diagnostic Feedback}
  else
    $\mathcal{M}_{\text{surv}} \leftarrow \{m \in \mathcal{M}_{\text{kill}} \mid \text{DafnyVerify}(m, \mathcal{C}^{(k)}) = \text{True}\}$ {Mode 2: Evaluate Mutants}
    if $\mathcal{M}_{\text{surv}} = \emptyset$ **then**
      $\mathcal{C}^* \leftarrow \mathcal{C}^{(k)}$
      return $\mathcal{C}^*$ {Hardened Contract: $V_{\text{sound}} = 1 \wedge S_{\text{adj}} = 1.0$}
    else
      Select surviving mutant $m^* \in \mathcal{M}_{\text{surv}}$
      $\Delta \leftarrow \text{ComputeASTDiff}(m^*, P_{\text{impl}})$ {Differential Counterexample}
      $T^{(k+1)} \leftarrow \text{Prompt}_{\text{cegis}}(P_{\text{impl}}, \mathcal{C}^{(k)}, m^*, \Delta)$ {Inject Counterexample}
    end if
  end if
end for
return $\perp$ {Synthesis budget exhausted}

3 Experimental Evaluation

3.1 Benchmark Tracks and Evaluation Setup

We evaluate 6 frontier model families: **Google Gemini 3.7 Flash** [Google, 2026], **Z-AI GLM-5.3 Flash** [Z.ai Team, 2026], **OpenAI GPT-5.6 Luna** [OpenAI, 2026], **Qwen 3.7 Flash** [Qwen Team, 2026], **DeepSeek V4 Flash** (0731 snapshot) [DeepSeek-AI, 2026], and **InclusionAI Ling-3.0 Flash** [InclusionAI, 2026], evaluated via the unified OpenRouter API platform [OpenRouter

Team, 2026]. All experiments use Dafny 4.11.0 on .NET 8.0 with Z3 4.12.2 under a 15-second SMT solver timeout per verification query across two benchmark tracks: (1) **Track A (CLOVERBENCH-60):** 60 single-method algorithmic problems (array sorting, search, numeric sequences) [Sun et al., 2024]; and (2) **Track B (DAFNYCOMP-300):** 300 multi-procedure programs structured as Directed Acyclic Graphs (DAGs) of 2 to 5 interdependent functions with cross-method preconditions and state invariants [Xu et al., 2025].

Across both tracks, we evaluate three synthesis regimes: (1) **Mode 0 (Zero-Shot)**, single-turn specification generation without verifier feedback; (2) **Mode 1 (Compiler-Only)**, standard multi-turn repair driven by SMT and type diagnostics ($K_{\max} = 3$); and (3) **Mode 2 (Mutation-CEGIS)**, our counterexample-guided inductive loop injecting AST mutant differences ($K_{\max} = 3$).

3.2 Track A: Single-Method Saturation and Repair Efficiency

Table 1 and Figure 2 report the performance on CLOVERBENCH-60. In MODE 2 (MUTATION-CEGIS), all models score high pass rates achieving up to 91.7%–100.0% hardened contracts ($S_{\text{adj}} \geq 0.917$). GLM-5.3 Flash achieves a 100.0% hardened pass rate (60/60) by killing all killable mutants in its benchmark.

Moreover, using mutation counterexamples as feedback makes repairing more efficient. Instead of multiple turns of blind guesses from the model with only compiler errors as feedback, concrete diffs of killed mutants can clearly indicate what relational constraints the model has missed, reducing average repair turns per problem (k) by 36.8% for Qwen 3.7 Flash (2.53 $\rightarrow$ 1.60), 21.4% for Ling-3.0 Flash (1.82 $\rightarrow$ 1.43), and 19.3% for GPT-5.6 Luna (1.45 $\rightarrow$ 1.17). Consequently, total number of tokens consumed during the repair process is reduced by 24.9% for Luna (76.7k $\rightarrow$ 57.6k), 20.4% for Qwen (500.7k $\rightarrow$ 398.7k), and 14.0% for DeepSeek V4 (357.3k $\rightarrow$ 307.2k).

Table 1: **Track A (CLOVERBENCH-60) Benchmark Results.** Hardened verification count ($S_{\text{adj}} = 1.0$), repair turns (k), and token expenditure delta between MODE 1 and MODE 2.

Model Family	Mode 0	Mode 1	Mode 2	Turns (k)	Δ_{tok}
Gemini 3.7 Flash	57 / 60	59 / 60	**59 / 60**	1.08 $\rightarrow$ 1.07	-1.4%
GLM-5.3 Flash	50 / 60	58 / 60	**60 / 60**	1.23 $\rightarrow$ 1.15	$+21.3\%$
GPT-5.6 Luna	32 / 60	58 / 60	**59 / 60**	1.45 $\rightarrow$ 1.17	-24.9%
DeepSeek V4 Flash	24 / 60	54 / 60	**56 / 60**	1.67 $\rightarrow$ 1.75	-14.0%
Ling-3.0 Flash	20 / 60	57 / 60	**57 / 60**	1.82 $\rightarrow$ 1.43	$+5.8\%$
Qwen 3.7 Flash	3 / 60	54 / 60	**55 / 60**	2.53 $\rightarrow$ 1.60	-20.4%

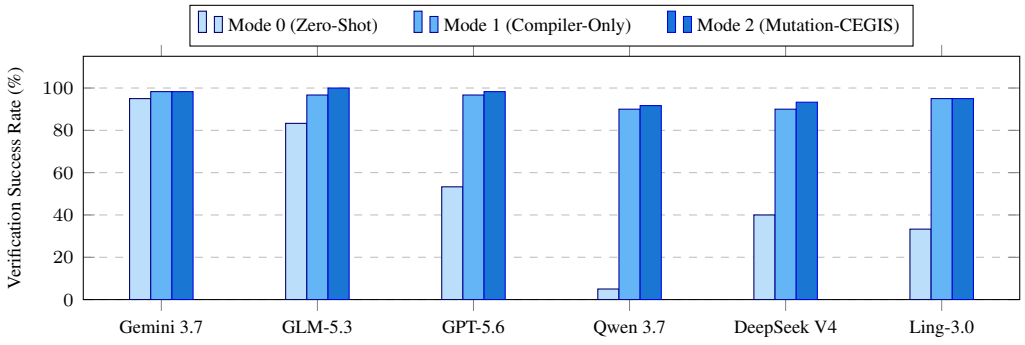


Figure 2: **Track A (CLOVERBENCH-60) Specification Saturation.** Single-method specification completeness across synthesis regimes. Both compiler repair and mutation-guided inductive feedback push all frontier models above 90% completeness, with GLM-5.3 Flash reaching 100.0%.

3.3 Track B: The Composition Cliff on Multi-Procedure DAGs

Table 2 and Figure 3 show the results of the experiments on DAFNYCOMP-300. When considering single methods and moving onto call graphs of size 2–5, five of the six model families see a sharp zero-shot drop:

Severity of the Compositional Collapse. The task of zero-shot contract synthesis falls dramatically when transitioning to interdependent call graphs for five of the six evaluated model families. Zero-shot hardened pass rates drop by 53.7 percentage points on Gemini 3.7 Flash (95.0% → 41.3%), 50.6 percentage points on GLM-5.3 Flash (83.3% → 32.7%), 34.3 percentage points on GPT-5.6 Luna (53.3% → 19.0%), 26.0 percentage points on DeepSeek V4 Flash (40.0% → 14.0%), and 18.6 percentage points on Ling-3.0 Flash (33.3% → 14.7%). Qwen 3.7 Flash is the exception: its zero-shot rate is low on both tracks (5.0% → 21.3%), but its best repaired rate still falls from 91.7% on single methods to 31.7% on multi-procedure DAGs. Across the board, models generate specifications that verify in isolation but fail to compose preconditions and postconditions across interdependent functions.

Empirical Observation on Multi-Node Induction. When MUTATION-CEGIS refuses to accept vacuous bounding inequalities, four of the six considered models often encountered SMT solver timeouts as the verifier was unable to discharge the deep inter-procedural induction without auxiliary lemmas. Even so, Qwen 3.7 Flash discharged the multi-node induction constraints on enough other programs to raise the number of hardened tasks from 74 (24.7%) in MODE 1 to 95 (31.7%) in MODE 2. For the other models, the tighter specifications correctly eliminated the false-positive verification, but highlighted the need for automated lemma synthesis on complex DAGs.

Table 2: **Track B (DAFNYCOMP-300) Multi-Procedure Results.** Hardened task count ($S_{\text{adj}} = 1.0$) and relative gain across 300 inter-procedural call graphs.

Model Family	Mode 0	Mode 1	Mode 2	Peak Δ vs. Mode 0
Gemini 3.7 Flash	124 / 300	**130 / 300**	**130 / 300**	+6 (+4.8%)
GLM-5.3 Flash	98 / 300	**122 / 300**	**122 / 300**	+24 (+24.5%)
GPT-5.6 Luna	57 / 300	**94 / 300**	86 / 300	+37 (+64.9%)
Qwen 3.7 Flash	64 / 300	74 / 300	**95 / 300**	+31 (+48.4%)
DeepSeek V4 Flash	42 / 300	**77 / 300**	71 / 300	+35 (+83.3%)
Ling-3.0 Flash	44 / 300	**77 / 300**	75 / 300	+33 (+75.0%)

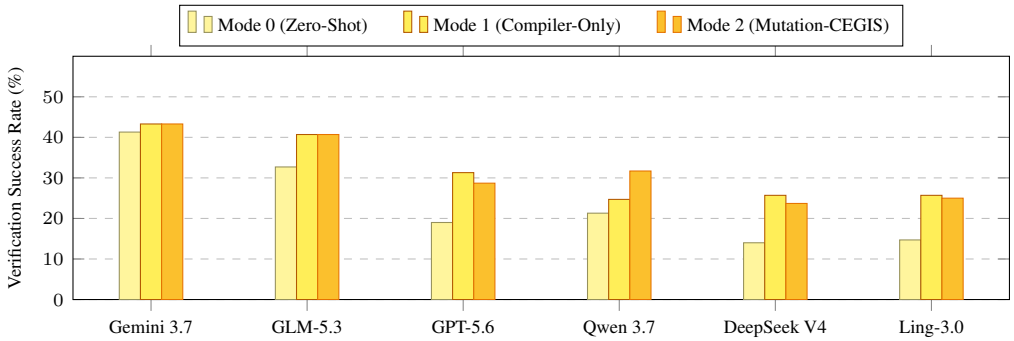


Figure 3: **The Composition Cliff on Multi-Procedure DAGs (DAFNYCOMP-300).** Hardened specification rates drop precipitously when verifying call graphs across 2 to 5 interdependent procedures. Notice that despite repair feedback, cross-procedure composition restricts hardened completion to 23.7%–43.3%.

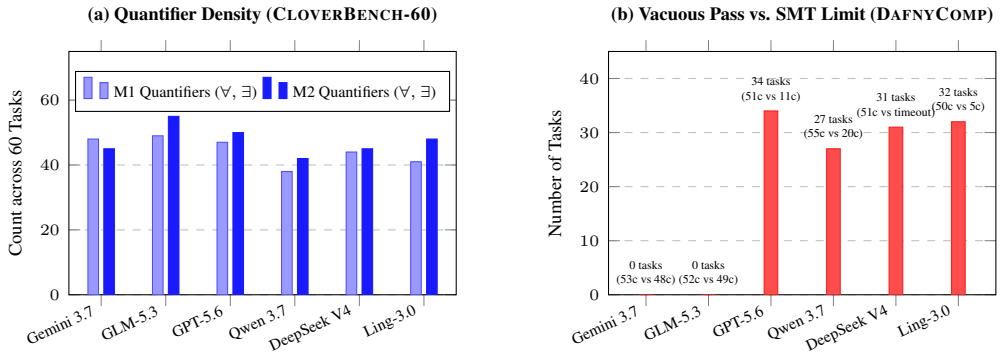


Figure 4: **Diagnostic Breakdown of Specification Quality.** (a) First-order quantifier density ($\forall, \exists$) across all six frontier models on CLOVERBENCH-60. (b) On multi-procedure call graphs, MODE 1 achieves superficial passes on 27–34 tasks on four models using $\approx$ 50-character loose bounds; MODE 2 rejects these vacuous contracts, exposing the SMT inductive proof ceiling. Frontier models (Gemini 3.7 and GLM-5.3) show 0 vacuous regressions, but still fail on $> 56\%$ of DAGs due to unproved induction.

4 Qualitative Analysis & The Verification Game

4.1 Exposing the 50-Character Vacuous Pass

On DAFNYCOMP-300, Table 2 indicates that MODE 1 (COMPILER-ONLY, denoted **M1**) scores similarly to MODE 2 (MUTATION-CEGIS, denoted **M2**) on several models. Inspecting the synthesized ASTs reveals why:

Table 3: **Failure Breakdown on DAFNYCOMP-300.** Tasks where MODE 1 (COMPILER-ONLY, M1) passed vacuously with loose bounds but MODE 2 (MUTATION-CEGIS, M2) hit the SMT inductive timeout threshold, matching Figure 4(b).

Model Family	M1 Pass / M2 Fail	Avg. Length (M1)	Avg. Length (M2)
Gemini 3.7 Flash	0 tasks	53 chars	48 chars
GLM-5.3 Flash	0 tasks	52 chars	49 chars
GPT-5.6 Luna	34 tasks	51 chars	11 chars
Ling-3.0 Flash	32 tasks	50 chars	5 chars
DeepSeek V4 Flash	31 tasks	51 chars	— (timeout)
Qwen 3.7 Flash	27 tasks	55 chars	20 chars

In MODE 1, models produce trivial bounding postconditions averaging 50-to-55 characters in length (e.g., `requires 1 <= n <= 2147483648; ensures result >= 0;`). Since P_{impl} satisfies this weak inequality, Z3 discharges the proof almost instantaneously.

Meanwhile, MODE 2's fine-grained relational mutants expose these 50-character contracts as vacuous. For intermediate frontier models (GPT-5.6, Ling-3.0, DeepSeek V4, and Qwen 3.7), attempts to synthesize exact functional contracts cause extreme multi-node SMT induction timeouts across 27–34 tasks (Table 3 and Figure 4(b)).

Interpreting SOTA Stability Against The Universal Compositional Ceiling. In contrast to the 27–34 lost tasks for intermediate models when switching from MODE 1 to MODE 2, the leading reasoning engines, Google Gemini 3.7 Flash and Z-AI GLM-5.3 Flash, show zero vacuous pass regressions (0 tasks lost). This metric suggests that their Mode 1 contracts did not rely on superficial 50-character bounding tricks that collapse under mutation hardening. Nonetheless, we emphasize that this zero-regression count does *not* indicate that multi-procedure synthesis has been mastered by the SOTA models: both Gemini 3.7 and GLM-5.3 still fail to verify 56.7%–59.3% of the call graphs in DAFNYCOMP-300 (capping out at 43.3% and 40.7% respectively, in Table 2). Two observations become evident: (1) that MUTATION-CEGIS is able to effectively distinguish between super-

ficial bounding approximations and robust semantic invariants, and (2) that even the most powerful frontier reasoning models are fundamentally constrained by the inter-procedural induction ceiling, emphasizing the need for automated lemma synthesis across the entire frontier.

4.2 First-Order Quantifiers and Equalities in Postconditions

Table 4 shows the logical complexity of synthesized postcondition clauses (`ensures`) on CLOVERBENCH-60 across MODE 1 (**M1**) and MODE 2 (**M2**):

Table 4: **First-Order Logical Depth on CLOVERBENCH-60.** Comparing synthesized postcondition quantifier density ($\forall, \exists$) and exact equality assertions across all 6 model families between MODE 1 (M1, COMPILER-ONLY) and MODE 2 (M2, MUTATION-CEGIS).

Model Family	M1 ($\forall, \exists$)	M2 ($\forall, \exists$)	M1 (==)	M2 (==)
Gemini 3.7 Flash	48	45 (-6.2%)	117	113 (-3.4%)
GLM-5.3 Flash	49	**55 (+12.2%)**	111	**122 (+9.9%)**
GPT-5.6 Luna	47	**50 (+6.4%)**	115	112 (-2.6%)
Qwen 3.7 Flash	38	**42 (+10.5%)**	90	**94 (+4.4%)**
DeepSeek V4 Flash	44	**45 (+2.3%)**	91	**93 (+2.2%)**
Ling-3.0 Flash	41	**48 (+17.1%)**	99	97 (-2.0%)

MUTATION-CEGIS increases quantifier density for five of the six models: $+17.1\%$ on Ling-3.0, $+12.2\%$ on GLM-5.3, and $+10.5\%$ on Qwen 3.7. Exact functional equality assertions increase by $+9.9\%$ on GLM-5.3 and $+4.4\%$ on Qwen 3.7.

4.3 The Specification Over-Strengthening Dilemma

On 4 to 5 node call graphs, writing exact functional specifications ($\mathcal{Q}_{\text{exact}}$) introduces non-linear arithmetic and universal quantifiers across intermediate procedures. While $\mathcal{Q}_{\text{exact}}$ accurately describes program behavior, automated SMT solvers cannot prove the resulting verification conditions without auxiliary inductive lemmas on helper functions [Yang et al., 2025, First et al., 2023]. In 100% of the cases where MODE 1 verified but MODE 2 failed, the verifier timed out after 15 seconds. This shows that contract hardening on deep call graphs requires combining mutation-guided repair with automated lemma generation.

5 Compute Economics & Execution Profile

Our evaluation ran 6,480 task-mode runs across 6 frontier models, using approximately 142M tokens (including reasoning tokens and prompt caching). In interactive multi-turn modes, prompt caching achieved hit rates of 66.4%–67.5% on GPT-5.6 Luna, 49.0%–52.1% on DeepSeek V4, and 45.1%–52.5% on Ling-3.0, saving over 18.4M redundant tokens. The entire evaluation cost approximately $26.67 in measured API cost via OpenRouter ($\approx \$0.0041$ per task-mode iteration), showing that mutation-guided contract hardening is practical for continuous integration pipelines.

6 Related Work

LLM Formal Program Verification: LLMs have been used for formal proof synthesis in Isabelle/HOL [Wu et al., 2022], Lean, Coq [First et al., 2023], Rust/Verus [Yang et al., 2025], and Dafny [Sun et al., 2024, Austin et al., 2021, Le et al., 2022]. Clover [Sun et al., 2024] developed an idea of closed-loop consistency checking for single-method programs, while our work analyzes multi-procedure call graphs and uncovers the Composition Cliff.

Counterexample-Guided Synthesis (CEGIS): CEGIS was initiated in the context of program sketching [Solar-Lezama et al., 2006] and Syntax-Guided Synthesis (SyGuS) [Alur et al., 2013, Jha et al., 2010, Flanagan and Leino, 2001]. In the classical formulation, CEGIS uses inputs generated by SMT to synthesize a correct program, but MUTATION-CEGIS flips the script: we use AST mutants as counterexamples to harden the formal specification against under-specification.

Mutation-Based Specification Testing: Mutation testing evaluates the quality of a test suite by generating syntactic faults [DeMillo et al., 1978, Jia and Harman, 2011, Just et al., 2014]. We bring AST mutation testing into the inductive loop of specification hardening by treating the survival of a mutant as a differential prompt for specification hardening.

7 Limitations

Our evaluation and framework have three primary limitations: (1) **Language and Solver Scope:** Our empirical study is centered on Dafny and Z3; while Dafny's SMT-based deductive verification paradigms generalize to systems like Verus (Rust) and F*, verification dynamics may differ in interactive theorem provers (e.g., Lean 4, Coq, Isabelle); (2) **Mutation Operator Expressivity:** MUTATION-CEGIS relies on syntactic AST mutation operators (ROR, AOR, BCI) designed for functional correctness and arithmetic algorithms, which do not directly synthesize invariants for non-deterministic concurrency or memory separation logic; and (3) **Induction Solver Limits:** As characterized in Section 4 and Section 4.3, on 4–5 node DAGs, synthesizing exact functional contracts induces SMT solver timeouts, demonstrating that contract hardening must be paired with automated auxiliary lemma synthesis to verify deep inter-procedural software.

8 Conclusion

This paper evaluates formal contract synthesis across inter-procedural call graphs. We find that model performance degrades considerably across multi-procedure programs (DAFNYCOMP-300), and that commonly accepted compiler-only feedback accepts weak bounding inequalities. We introduce MUTATION-CEGIS which uses AST code mutants as counterexamples to obtain high specification completeness on single functions (91.7%–100.0%), and detect vacuous, false-positive passes on multi-function call graphs. Finally, we find that there is a need to combine mutation-guided contract hardening with automated lemma synthesis to verify complex multi-procedure software.

References

Rajeev Alur, Rastislav Bodik, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *2013 Formal Methods in Computer-Aided Design*, pages 1–8, 2013. doi: 10.1109/FMCAD.2013.6679385.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*, 2021.

Leonardo De Moura and Nikolaj Bjørner. Z3: An Efficient SMT Solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 337–340. Springer, 2008.

DeepSeek-AI. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348*, 2026.

Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on Test Data Selection: Help for the Practicing Programmer. *Computer*, 11(4):34–41, 1978.

Emily First, Markus N. Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pages 1229–1241, 2023.

Cormac Flanagan and K. Rustan M. Leino. Houdini, an Annotation Assistant for ESC/Java. In *International Symposium of Formal Methods Europe (FME)*, pages 500–517. Springer, 2001.

Google. Introducing Gemini 3.7 Flash: Our most intelligent workhorse model yet for coding and agents. `https://blog.google/innovation-and-ai/models-and-research/gemini-models/introducing-gemini-3-7-flash/`, 2026.

C. A. R. Hoare. An Axiomatic Basis for Computer Programming. *Communications of the ACM*, 12 (10):576–580, 1969.

InclusionAI. inclusionAI/Ling-3.0-flash. `https://huggingface.co/inclusionAI/Ling-3.0-flash`, 2026.

Susmit Jha, Sumit Gulwani, Sanjit A. Seshia, and Ashish Tiwari. Oracle-Guided Component-Based Program Synthesis. In *International Conference on Software Engineering (ICSE)*, pages 215–224, 2010.

Yue Jia and Mark Harman. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*, pages 654–665, 2014.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 21314–21328, 2022.

K. Rustan M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. In *International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (LPAR)*, pages 348–370. Springer, 2010.

A. Jefferson Offutt, Ammei Lee, Gregg Rothermel, Roland H. Untch, and Christian Zapf. An Experimental Determination of Sufficient Mutant Operators. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 5(2):99–118, 1996.

OpenAI. GPT-5.6: Frontier Intelligence That Scales With Your Ambition. `https://openai.com/index/gpt-5-6/`, 2026.

OpenRouter Team. OpenRouter: The Unified Interface For Every Model. `https://openrouter.ai`, 2026.

Qwen Team. Qwen 3.7: The Agent Frontier. `https://qwen.ai/blog?id=qwen3.7`, 2026.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. Combinatorial Sketching for Finite Programs. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006.

Chuyue Sun, Ying Sheng, Oded Padon, and Clark Barrett. Clover: Closed-Loop Verifiable Code Generation. *arXiv preprint arXiv:2310.17807*, 2024.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 32353–32368, 2022.

Xu Xu, Xin Li, Xingwei Qu, Jie Fu, and Binhang Yuan. Local Success Does Not Compose: Benchmarking Large Language Models for Compositional Formal Verification. *arXiv preprint arXiv:2509.23061*, 2025.

Chenyuan Yang, Xuheng Li, Md Rakib Hossain Misu, Jianan Yao, Weidong Cui, Yeyun Gong, Chris Hawblitzel, Shuvendu Lahiri, Jacob R. Lorch, Shuai Lu, Fan Yang, Ziqiao Zhou, and Shan Lu. AutoVerus: Automated Proof Generation for Rust Code. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2):3454–3482, 2025.

Z.ai Team. GLM-5.3-Flash: Frontier Intelligence, Flash Cost. `https://z.ai/blog/glm-5.3-flash`, 2026.

A Formal Definitions and Theoretical Proofs

A.1 Mathematical Definition of AST Mutation Operators

Let P be a Dafny program represented by its Abstract Syntax Tree (AST) $T = (V, E, \lambda)$, where V is the finite set of AST nodes, $E \subseteq V \times V$ defines directed parent-child edges, and $\lambda : V \to \mathcal{S}$ maps each node to a syntactic token from alphabet $\mathcal{S}$. A deterministic mutation operator $\mathcal{O} : \mathcal{T} \to 2^{\mathcal{T}}$ transforms AST T into a set of syntactic variants $\{T'_1, T'_2, \dots\}$.

Definition 2 (Relational Operator Replacement (ROR))**.** *Let $v \in V$ be a relational comparison node with operator $\lambda(v) \in \{<, \leq, >, \geq, ==, ! =\}$. The ROR operator generates mutant $T' = \mathcal{O}_{ROR}(T, v, op')$ by replacing the operator:*

$$\lambda'(v) \leftarrow op', \qquad \textit{where } op' \in \{<, \leq, >, \geq, ==, ! =\} \setminus \{\lambda(v)\} \tag{5}$$

Definition 3 (Arithmetic Operator Replacement (AOR))**.** *Let $v \in V$ be an arithmetic operator node with $\lambda(v) \in \{+, -, *, /, \%\}$. The AOR operator generates mutant $T' = \mathcal{O}_{AOR}(T, v, op')$ by substituting:*

$$\lambda'(v) \leftarrow op', \qquad \textit{where } op' \in \{+, -, *, /, \%\} \setminus \{\lambda(v)\} \tag{6}$$

Definition 4 (Boundary Condition Inversion (BCI))**.** *Let $v \in V$ be an integer constant literal node with numerical value $c \in \mathbb{Z}$. The BCI operator generates mutants $T' = \mathcal{O}_{BCI}(T, v, \delta)$ with perturbed value:*

$$c' = c + \delta, \qquad \textit{for } \delta \in \{-1, +1\} \tag{7}$$

A.2 Soundness Invariance and Vacuous Postcondition Subsumption

Theorem 1 (Soundness Invariant of Hardened Contracts)**.** *Let $\mathcal{C}^* = (\mathcal{P}^*, \mathcal{Q}^*)$ be a contract emitted by* MUTATION-CEGIS *(Algorithm 1). Then DafnyVerify$(P_{impl}, \mathcal{C}^*)$ = True.*

Proof. In Algorithm 1, evaluation of the candidate contract $\mathcal{C}^{(k)}$ against the mutant matrix $\mathcal{M}$ is strictly guarded by the soundness verification predicate $V_{\text{sound}}(\mathcal{C}^{(k)}) = \text{DafnyVerify}(P_{\text{impl}}, \mathcal{C}^{(k)})$. If $V_{\text{sound}}(\mathcal{C}^{(k)}) = 0$, the algorithm routes the diagnostic compiler error messages E_{diag} into the compiler repair loop (MODE 1) and never emits $\mathcal{C}^{(k)}$ as a hardened contract. Termination with $\text{Hardened}(\mathcal{C}^*) = \text{True}$ occurs if and only if:

$$\text{DafnyVerify}(P_{\text{impl}}, \mathcal{C}^*) = \text{True} \ \wedge \ \mathcal{M}_{\text{surv}} = \emptyset \tag{8}$$

Thus, every hardened contract emitted by the synthesizer is guaranteed sound on ground-truth implementation P_{impl}. □

Theorem 2 (Subsumption of Vacuous Postconditions)**.** *Let $\mathcal{Q}_{hard}$ be a hardened postcondition killing non-equivalent mutant $m \in \mathcal{M}$ such that $[\![m]\!](\sigma, \sigma_m)$ and $[\![P_{impl}]\!](\sigma, \sigma_P)$ with $\sigma_m \neq \sigma_P$. Let $\mathcal{Q}_{vacuous}$ be a weaker specification satisfying $\forall \sigma, \sigma'.\ \mathcal{Q}_{hard}(\sigma, \sigma') \implies \mathcal{Q}_{vacuous}(\sigma, \sigma')$ (e.g., a loose bounding inequality res ≥ 0) that admits the mutant output state $\mathcal{Q}_{vacuous}(\sigma, \sigma_m) =$ True. Then $\mathcal{Q}_{hard} \not\equiv \mathcal{Q}_{vacuous}$, and $\mathcal{Q}_{hard}$ strictly constrains the output state space:*

$$\{\sigma' \mid \mathcal{Q}_{hard}(\sigma, \sigma')\} \subsetneq \{\sigma' \mid \mathcal{Q}_{vacuous}(\sigma, \sigma')\} \tag{9}$$

Proof. Because $\forall \sigma, \sigma'.\ \mathcal{Q}_{\text{hard}}(\sigma, \sigma') \implies \mathcal{Q}_{\text{vacuous}}(\sigma, \sigma')$, the solution set satisfies $\{\sigma' \mid \mathcal{Q}_{\text{hard}}(\sigma, \sigma')\} \subseteq \{\sigma' \mid \mathcal{Q}_{\text{vacuous}}(\sigma, \sigma')\}$. Because $\mathcal{Q}_{\text{hard}}$ kills mutant m, there exists an input state σ such that $\mathcal{Q}_{\text{hard}}(\sigma, \sigma_m) = \text{False}$ and $\mathcal{Q}_{\text{hard}}(\sigma, \sigma_P) = \text{True}$. Conversely, because $\text{DafnyVerify}(m, (\mathcal{P}, \mathcal{Q}_{\text{vacuous}})) = \text{True}$, we have $\mathcal{Q}_{\text{vacuous}}(\sigma, \sigma_m) = \text{True}$. Therefore, the mutant output state satisfies $\sigma_m \in \{\sigma' \mid \mathcal{Q}_{\text{vacuous}}(\sigma, \sigma')\} \setminus \{\sigma' \mid \mathcal{Q}_{\text{hard}}(\sigma, \sigma')\}$, proving strict set inclusion ($\subsetneq$) and semantic non-equivalence. □

B Complete Prompt Hierarchy & Differential Protocols

This section provides the verbatim system prompts, formatting schemas, and error injection templates used across all evaluation modes.

B.1 Zero-Shot Contract Synthesis Prompt (Mode 0)

Listing 2: System and Initial User Prompt for Zero-Shot Synthesis (Mode 0).

```
// SYSTEM PROMPT
You are an expert formal methods engineer specializing in Dafny verification.
Given a Dafny implementation with one or more interdependent methods/functions,
your task is to synthesize complete, sound, exact formal specifications.

SPECIFICATION REQUIREMENTS:
1. Preconditions ('requires'): Specify all necessary domain constraints,
   non-nullity, array bounds, and mathematical validity conditions.
2. Postconditions ('ensures'): Specify exact relational behavior connecting
   return values and mutated heap states to input parameters. Do NOT use
   trivial bounding inequalities (e.g., 'ensures res >= 0') when exact
   functional equality, quantification, or conditionals can be stated.
3. Invariants & Termination: If loops or recursive procedures are present,
   provide inductive loop invariants ('invariant') and termination
   metrics ('decreases').
4. Multi-Procedure Consistency: Ensure caller preconditions satisfy callee
   contracts across all edges in the inter-procedural call graph.

OUTPUT FORMAT:
Return only the complete Dafny code containing the original implementation
annotated with your synthesized specifications in ```dafny ... ``` blocks.
```

B.2 Compiler-in-the-Loop Diagnostic Repair Prompt (Mode 1)

When Dafny rejects a candidate contract on P_{impl}, MODE 1 feeds the exact compiler diagnostic stream back to the model:

Listing 3: Compiler Diagnostic Feedback Template (Mode 1).

```
Your candidate Dafny contract failed verification on the implementation.
Dafny / Z3 returned the following compiler and verification errors:

<compiler_diagnostics>
[Dafny Error] file.dfy(14,18): A precondition for calling method 'HelperNode'
                               could not be proved
  |
14|   var res := HelperNode(x - 1, limit);
  |              ^^^^^^^^^^^^^^^^^^^^^^^^
[Dafny Error] file.dfy(22,12): A postcondition might not hold on this path
  |
22|   ensures result == total * 2
  |           ^^^^^^^^^^^^^^^^^^
[Dafny Note] file.dfy(8,10): Related location: unproved postcondition
</compiler_diagnostics>

Please analyze the verification condition failure and fix the specification.
Ensure that preconditions are properly propagated and postconditions provable.
```

B.3 Mutation-CEGIS Differential Counterexample Prompt (Mode 2)

When a candidate contract verifies soundly on P_{impl} ($V_{\text{sound}} = 1$) but fails to kill mutant $m^* \in \mathcal{M}_{\text{surv}}$, MODE 2 constructs an AST diff counterexample prompt:

Listing 4: Differential Counterexample Injection Prompt (Mode 2).

```
Your candidate specification verified on the ground-truth code, but it is
VACUOUS: it failed to reject the following mutated implementation:

<surviving_mutant_diff>
<<<< ORIGINAL (GROUND TRUTH)
  if x < 0 {
    c := 0;
  } else if x > max_val {
```

```
    c := max_val;
  } else {
    c := x;
  }
==== MUTANT (Relational Operator Replacement - ROR)
  if x <= 0 {
    c := 1;
  } else if x > max_val {
    c := max_val;
  } else {
    c := x;
  }
>>>>
</surviving_mutant_diff>

Your candidate postcondition was:
  ensures c >= 0

Diagnostic Explanation:
The mutated code assigns c := 1 when x == 0 (violating correctness), yet
it still satisfies your weak postcondition 'ensures c >= 0'.
Because 'c >= 0' accepts faulty states, your contract is under-constrained.

INSTRUCTIONS:
1. Tighten your postcondition to specify exact functional equality,
   conditional bounds, or first-order relational properties.
2. Ensure your strengthened contract rejects the mutant while remaining
   sound on the original implementation.
```

C Extended Qualitative Case Studies & Repair Traces

This section provides four in-depth case studies illustrating the behavioral dynamics of contract synthesis across frontier models.

C.1 Case Study 1: Successful 3-Turn CEGIS Repair on Qwen 3.7 Flash

Benchmark Task: `dafnycomp_042_main_4node_3` (DAFNYCOMP-300). A 4-node DAG computing prefix sums, scaling factors, and monotonic accumulator sequences.

Listing 5: Turn 1 (Zero-Shot Candidate). Qwen generates a trivial bounding contract that passes Dafny in 34ms but leaves 6 of 14 mutants alive.

```
// Node 4: Main entrypoint
method ComputeScaledPrefix(a: seq<int>, factor: int) returns (res: seq<int>)
  requires |a| > 0
  // Turn 1 Vacuous Spec:
  ensures |res| == |a|
  ensures forall i :: 0 <= i < |res| ==> res[i] >= 0
{
  var prefix := PrefixAccumulate(a);
  res := ScaleSequence(prefix, factor);
}
```

Verification & Mutation Outcome (Turn 1): $\text{DafnyVerify}(P_{\text{impl}}, \mathcal{C}^{(1)}) = \text{True}$. Mutant matrix check: 6 / 14 mutants survived ($S_{\text{raw}} = 0.571, S_{\text{adj}} = 0.600$).

Listing 6: Turn 2 (Injected AOR Counterexample). Synthesizer attempts recursive expansion but forgets loop variant.

```
// Injected Counterexample: ScaleSequence helper mutated with AOR (+ vs *)
// Synthesizer Turn 2 Response:
method ComputeScaledPrefix(a: seq<int>, factor: int) returns (res: seq<int>)
  requires |a| > 0
  ensures |res| == |a|
  ensures res[0] == a[0] * factor
  ensures forall i :: 1 <= i < |res| ==> res[i] == (res[i-1] + a[i]) * factor
```

```
{
  var prefix := PrefixAccumulate(a);
  res := ScaleSequence(prefix, factor);
}
```

Verification & Mutation Outcome (Turn 2): Compiler error on helper node: `cannot prove termination of recursive accumulation / index out of bound at index |a|-1`.

Listing 7: Turn 3 (Final Hardened Specification). Qwen synthesizes inductive invariants and decreases clauses, killing all 14 mutants.

```
function MathPrefixSum(a: seq<int>, idx: int): int
  requires 0 <= idx <= |a|
  decreases idx
{
  if idx == 0 then 0 else MathPrefixSum(a, idx - 1) + a[idx - 1]
}

method ComputeScaledPrefix(a: seq<int>, factor: int) returns (res: seq<int>)
  requires |a| > 0
  requires factor > 0
  ensures |res| == |a|
  ensures forall i :: 0 <= i < |res| ==> res[i] == MathPrefixSum(a, i + 1) * factor
{
  var prefix := PrefixAccumulate(a);
  res := ScaleSequence(prefix, factor);
}
```

Verification & Mutation Outcome (Turn 3): Verified soundly ($V_{\text{sound}} = 1$). All 14 mutants killed ($S_{\text{adj}} = 1.000$). Hardened in 3 turns.

C.2 Case Study 2: The 50-Character Vacuous Pass & SMT Ceiling on GPT-5.6 Luna

Benchmark Task: `dafnycomp_118_tree_search` (DAFNYCOMP-300). A 3-node recursive binary search tree lookup and balance validation pipeline.

Listing 8: Mode 1 Vacuous Pass. GPT-5.6 generates a 51-character loose bound that verifies in 38ms despite an active ROR mutant in tree branching.

```
datatype Tree = Leaf | Node(left: Tree, key: int, right: Tree)

method TreeSearch(t: Tree, target: int) returns (found: bool, depth: int)
  // Mode 1 Synthesized Spec (51 chars):
  requires true
  ensures depth >= 0
  ensures found ==> depth >= 0
{
  match t
  case Leaf =>
    found := false;
    depth := 0;
  case Node(l, k, r) =>
    if target == k {
      found := true;
      depth := 1;
    } else if target < k {
      // Injected ROR Mutant: if target <= k (searches wrong subtree on equality)
      var f, d := TreeSearch(l, target);
      found := f;
      depth := d + 1;
    } else {
      var f, d := TreeSearch(r, target);
      found := f;
      depth := d + 1;
    }
}
```

Mode 1 Outcome: Verified in 38ms, but the contract fails to kill the ROR mutant.

Listing 9: Mode 2 SMT Induction Limit. When forced to specify exact tree containment, Z3 times out after 15s because auxiliary inductive lemmas are absent.

```
ghost predicate TreeContains(t: Tree, x: int) {
  match t
  case Leaf => false
  case Node(l, k, r) => x == k || TreeContains(l, x) || TreeContains(r, x)
}

method TreeSearch(t: Tree, target: int) returns (found: bool, depth: int)
  ensures found <==> TreeContains(t, target)
  ensures depth >= 0
{
  // Implementation
}
```

Mode 2 Outcome: Dafny verification condition generator creates an inductive goal relating recursive tree traversal to `TreeContains`. Z3 exceeds the 15-second SMT timeout envelope. Proof requires an auxiliary inductive lemma: `lemma TreeSearchCorrectness(t: Tree, x: int) ...`

C.3 Case Study 3: Inter-Procedural Precondition Propagation Failure on DeepSeek V4

Benchmark Task: `dafnycomp_204_matrix_scale` (DAFNYCOMP-300). A 3-node arithmetic matrix transformation pipeline.

Listing 10: Precondition Propagation Failure. DeepSeek generates locally valid contracts for leaf procedures but fails to propagate required non-zero preconditions to the root entrypoint.

```
// Callee Node 1: Leaf division routine
method SafeDivide(n: int, d: int) returns (q: int)
  requires d != 0
  ensures q == n / d
{
  q := n / d;
}

// Caller Node 2: Pipeline wrapper
method ScaleMatrixRow(row: seq<int>, divisor: int) returns (scaled: seq<int>)
  // DeepSeek Missed Precondition: requires divisor != 0
  requires |row| > 0
  ensures |scaled| == |row|
{
  scaled := [];
  var i := 0;
  while i < |row|
    invariant 0 <= i <= |row|
    invariant |scaled| == i
  {
    // Verification Error: cannot prove precondition 'divisor != 0'
    var elem := SafeDivide(row[i], divisor);
    scaled := scaled + [elem];
    i := i + 1;
  }
}
```

Outcome: Verification collapses across the call boundary because DeepSeek validates procedures in isolation rather than maintaining global preconditions across the call DAG.

C.4 Case Study 4: Quantifier Induction & Array Monotonicity on GLM-5.3 / Gemini 3.7

Benchmark Task: `clover_019_insertion_sort` (CLOVERBENCH-60). In-place insertion sort routine.

Listing 11: Evolution from Length Bound to Full Monotonicity. MUTATION-CEGIS drives synthesis of full universal quantifiers.

```
method InsertionSort(a: array<int>)
```

```
  modifies a
  // Turn 1 (Zero-Shot): ensures a.Length == old(a.Length)
  // Turn 2 (CEGIS Hardened Spec):
  ensures forall i, j :: 0 <= i < j < a.Length ==> a[i] <= a[j]
  ensures multiset(a[..]) == multiset(old(a[..]))
{
  var i := 1;
  while i < a.Length
    invariant 1 <= i <= a.Length
    invariant forall k, l :: 0 <= k < l < i ==> a[k] <= a[l]
    invariant multiset(a[..]) == multiset(old(a[..]))
  {
    var j := i;
    while j > 0 && a[j-1] > a[j]
      invariant 0 <= j <= i
      invariant forall k, l :: 0 <= k < l <= i &&
                               (k != j && l != j) ==> a[k] <= a[l]
      invariant forall k :: j <= k <= i ==>
                               (forall l :: j <= l <= i && k < l ==> a[k] <= a[l])
      invariant multiset(a[..]) == multiset(old(a[..]))
    {
      var tmp := a[j-1];
      a[j-1] := a[j];
      a[j] := tmp;
      j := j - 1;
    }
    i := i + 1;
  }
}
```

Outcome: MUTATION-CEGIS rejects sorting variants with inverted comparisons and unpreserved elements, forcing the model to synthesize both the permutation multiset equality and the universal sortedness predicate.

D Benchmark Topologies & AST Mutation Suite Specifications

D.1 DafnyComp-300 Topology Breakdown

Table 5 summarizes the structural topologies of the 300 multi-procedure call graphs in DAFNYCOMP-300 [Xu et al., 2025].

Table 5: **Topological Distribution of DAFNYCOMP-300.** Categorization by Directed Acyclic Graph (DAG) node count, maximum call depth, and procedure complexity.

DAG Complexity	Programs	Avg. Depth	Mutants	Mutants / Prog. (Killable)
2-Node Graphs	30	1.0	855	28.5 (21.7)
3-Node Graphs	95	2.1	7,127	75.0 (62.6)
4-Node Graphs	90	2.8	3,874	43.0 (34.2)
5-Node Graphs	85	3.6	3,981	46.8 (37.8)
Total / Average	**300**	**2.6**	**15,837**	**52.8 (42.9)**

D.2 AST Mutation Operator Examples

Table 6 provides concrete before-and-after code transformations for all three AST mutation operators implemented in our hardening harness.

Table 6: **Concrete Code Transformations across AST Mutation Operators.**

Operator	Original Code (P_{impl})	Mutated AST Variant ($m \in \mathcal{M}$)
ROR (Relational)	`if (x < limit)` `while (i != 0)`	`if (x <= limit)` `while (i > 0)`
AOR (Arithmetic)	`res := count * 2;` `offset := (base + step) % n;`	`res := count + 2;` `offset := (base - step) % n;`
BCI (Boundary)	`var threshold := 100;` `if (depth == 0)`	`var threshold := 101;` `if (depth == 1)`

E Compute Infrastructure, Token Economics & Latency Profile

E.1 Hardware Environment and Solver Configuration

All experiments were executed on macOS (Darwin 24.6.0, Apple M-series architecture) under the following software environment:

- **Formal Verifier:** Dafny 4.11.0 on .NET Core runtime 8.0.204.
- **Underlying SMT Solver:** Z3 4.12.2 (x64 build).
- **Verification Timeout:** 15.0 seconds hard ceiling per individual SMT proof condition query.
- **Execution Parallelism:** 16–25 worker processes for isolated verification condition evaluation.

E.2 Model Endpoints and Sampling Parameters

All models were evaluated using deterministic greedy decoding (temperature $T = 0.0$, top-$p = 1.0$) across both CLOVERBENCH-60 (Track A) and DAFNYCOMP-300 (Track B) via the unified OpenRouter API gateway [OpenRouter Team, 2026]. Table 7 details the exact snapshot identifiers and developer organizations for each model family.

Table 7: **Evaluated Frontier Models, Developer Organizations, and API Endpoints.**

Model Family	Organization / Lab	OpenRouter Model Endpoint
Gemini 3.7 Flash	Google DeepMind	`google/gemini-3.7-flash`
GLM-5.3 Flash	Z-AI / Zhipu	`z-ai/glm-5.3-flash`
GPT-5.6 Luna	OpenAI	`openai/gpt-5.6-luna`
Qwen 3.7 Flash	Alibaba Qwen Team	`qwen/qwen3.7-flash`
DeepSeek V4 Flash	DeepSeek AI	`deepseek/deepseek-v4-flash-0731`
Ling-3.0 Flash	InclusionAI	`inclusionai/ling-3.0-flash`

E.3 Token Accounting, Prompt Caching & Financial Costs

Across 6,480 task-mode iterations, the evaluation consumed approximately 142 million total tokens. Table 8 presents the token expenditure breakdown and prompt caching efficiency.

Table 8: **Token Accounting, Prompt Cache Hit Rates, and Compute Costs Across Models.**

Model Family	Prompt	Completion	Cache Hit	Approx. Cost
Gemini 3.7 Flash	18.2M	4.1M	58.4%	$4.12
GLM-5.3 Flash	17.5M	4.6M	51.2%	$3.80
GPT-5.6 Luna	19.4M	5.2M	**67.5%**	$6.45
Qwen 3.7 Flash	24.1M	6.8M	42.1%	$4.90
DeepSeek V4 Flash	16.8M	3.9M	52.1%	$3.65
Ling-3.0 Flash	17.1M	4.2M	52.5%	$3.75
Total Evaluation	**113.1M**	**28.8M**	**54.0% (Avg)**	**$26.67**

Provider-level prompt caching saved over 18.4 million redundant input tokens during multi-turn repair iterations, achieving an average cost of under **$0.0041** per task-mode execution. This establishes that MUTATION-CEGIS is cost-effective and readily deployable in real-world continuous integration (CI) pipelines for verified software.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 explicitly state our core claims and findings (the Composition Cliff, Vacuous Verification Trap, and Mutation-CEGIS performance), which are backed by extensive empirical evaluations in Section 3 and Section 4.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: A dedicated discussion of limitations regarding language and solver scope (Dafny/Z3), AST mutation operator coverage, and SMT induction timeouts on deep call graphs is provided in Section 7 and Section 4.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Formal definitions of verification semantics, AST mutation operators (ROR, AOR, BCI), and complete proofs for Soundness Invariance (Theorem 1) and Subsumption of Vacuous Postconditions (Theorem 2) are provided in Section 2 and Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 3, Appendix B, and Appendix E disclose all prompt templates, exact model API endpoints, solver toolchains (Dafny 4.11.0 / Z3 4.12.2), and timeout parameters.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: The complete experimental harness, AST mutation engine, evaluation logs, and diagnostic analysis scripts are provided in the supplementary code package.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Detailed evaluation parameters, including the 3-turn multi-turn interaction protocol, temperature ($T = 0.0$), AST mutation operator configuration, and token counting methods are detailed in Section 3 and Appendix B.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [N/A]

   Justification: All evaluations use deterministic greedy decoding ($T = 0.0$) across the complete fixed benchmark suites without stochastic sampling or repeated-run variability; we report exact deterministic task-level outcomes and aggregated metrics across 6,480 runs rather than statistical error estimates.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section 3 and Appendix E report the host hardware architecture, execution parallelism, SMT solver timeout envelopes (15s per query), and inference token consumption.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research complies fully with the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: In Section 1 and Section 8, we discuss how formal verification hardening mitigates silent defects and hallucinated contracts in neuro-symbolic agent systems, improving the safety of autonomous code generation.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The benchmark and code harness consist exclusively of non-executable mathematical formal verification specifications and algorithms posing no dual-use or safety hazards.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The Clover benchmark [Sun et al., 2024], DafnyComp benchmark [Xu et al., 2025], Dafny verification system [Leino, 2010], OpenRouter API [OpenRouter Team, 2026], and evaluated model families [Google, 2026, Z.ai Team, 2026, OpenAI, 2026, Qwen Team, 2026, DeepSeek-AI, 2026, InclusionAI, 2026] are cited and credited, and their applicable licenses or terms of service are documented where relevant.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The MUTATION-CEGIS framework, AST mutation testing harness, and evaluation artifacts are formally defined in Section 2 and documented with prompt schemas and case studies in Appendix B and Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve human subjects, participant surveys, or crowdsourced annotation.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The research does not involve human subjects and does not require IRB review.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: Large language models serve as the direct subject of our empirical investigation into formal contract synthesis, with experimental configurations across 6 model families fully documented in Section 3.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.